DuckieTown: Perceptual Robustness

Bachelor Practical Course

Aditi Deshpande, Tannu Malik

Project Overview

Study robustness of the Duckietown lane-following pipeline

→

Analyze how perception errors affect robot motion

→

Test system under visual and system disturbances

→

Extend pipeline with extra perception functions

What is Perceptual Robustness?

Perceptual robustness refers to the ability of a system to maintain **stable and accurate perception** despite changes or disturbances in the environment.

In Duckietown context:

- The robot must reliably detect lane markings under:
  - Different lighting conditions
  - Shadows
  - Noise or visual disturbances

Key idea:
Even if the control system is correct, **poor perception leads to wrong decisions and unstable motion**.

Why Perceptual Robustness Matters

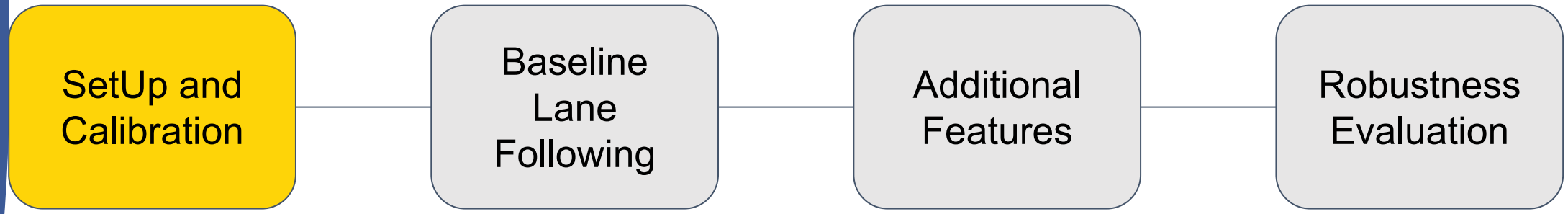
Autonomous driving depends heavily on perception
Small visual changes can cause large behavior errors
Errors propagate through the pipeline:

- Detection → Estimation → Control

A robust system:

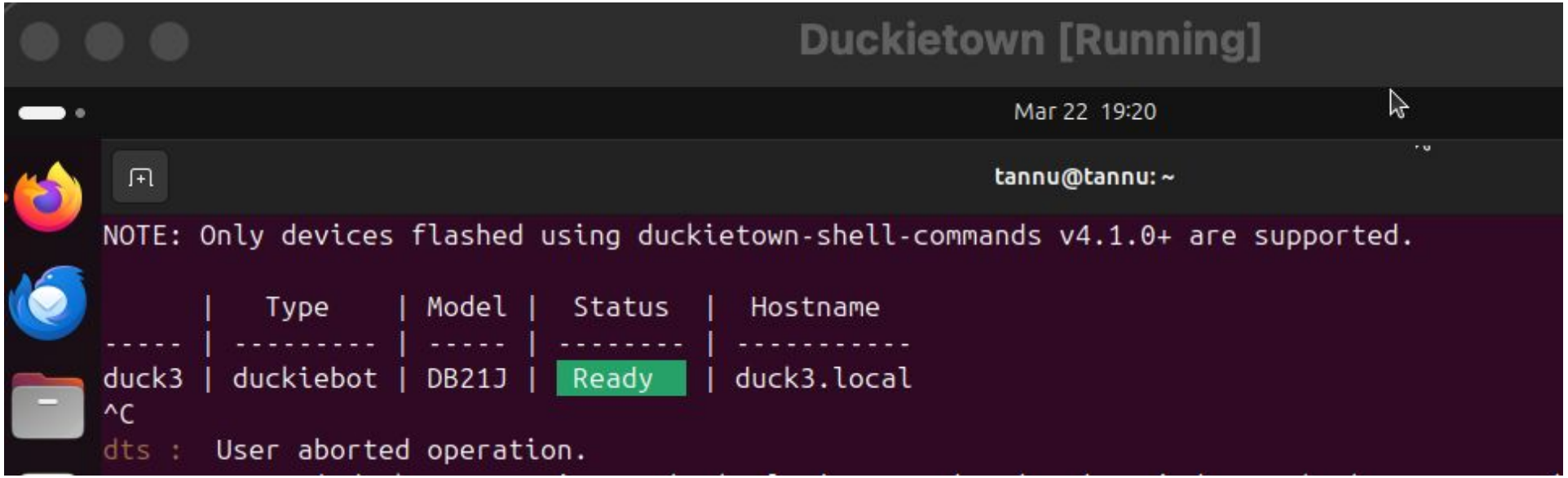
- Improves safety
- Reduces failure cases
- Enables reliable real-world deployment

Milestones

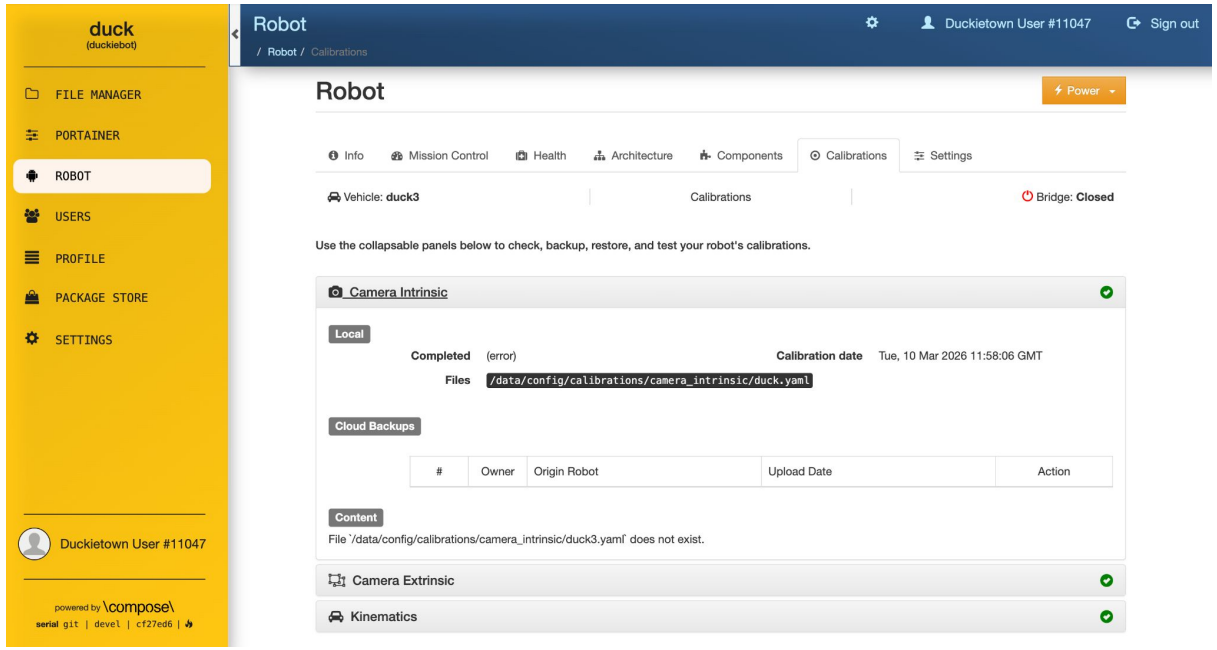

SetUp and Calibration
Baseline Lane Following
Additional Features
Robustness Evaluation

Setup

- Install and Setup VM and accounts
- DTS Installation
- Preparing the environment by installing necessary dependencies
- Flashing the SD card
- Running basic Duckietown functionalities from the manual

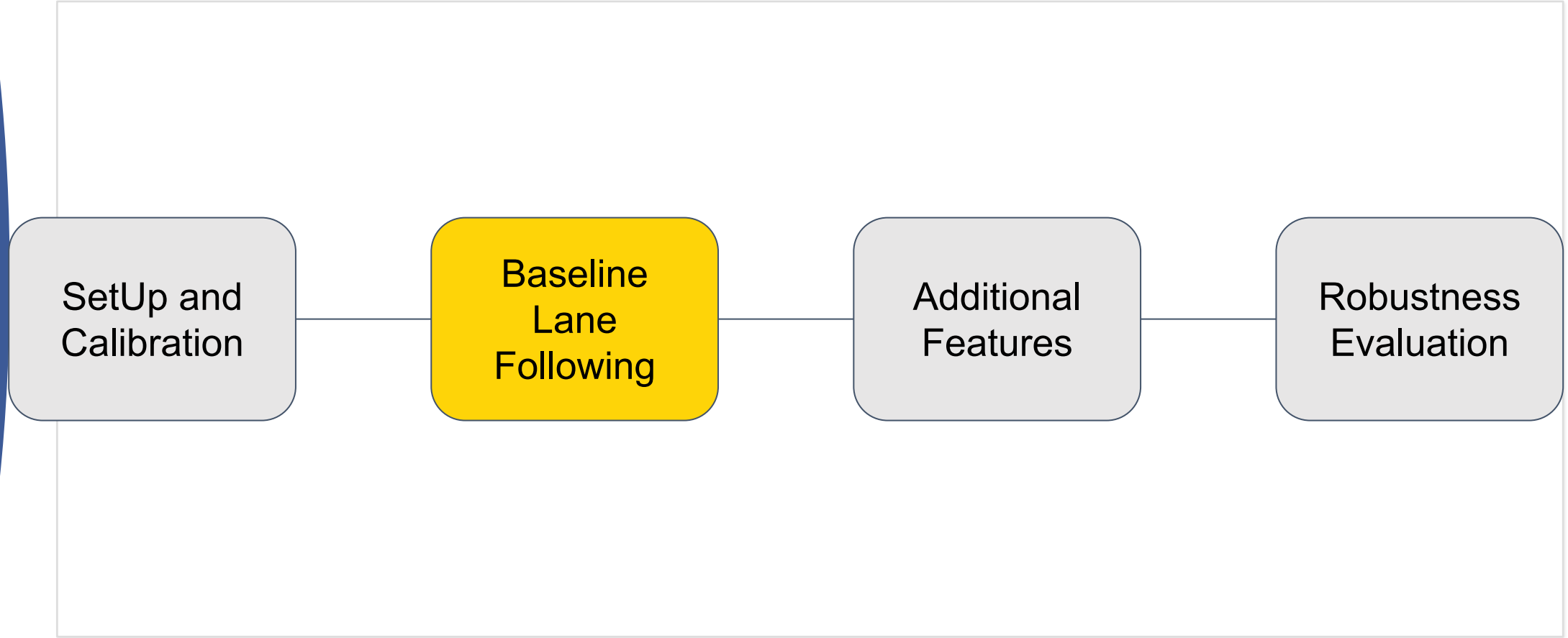


Callibration Steps



- Camera intrinsics: improve image accuracy and reduce distortion
- Camera extrinsics: enable correct ground projection
- Wheel Calibration: improve straight driving and motion consistency

Milestones


SetUp and Calibration
Baseline Lane Following
Additional Features
Robustness Evaluation

Lane Following

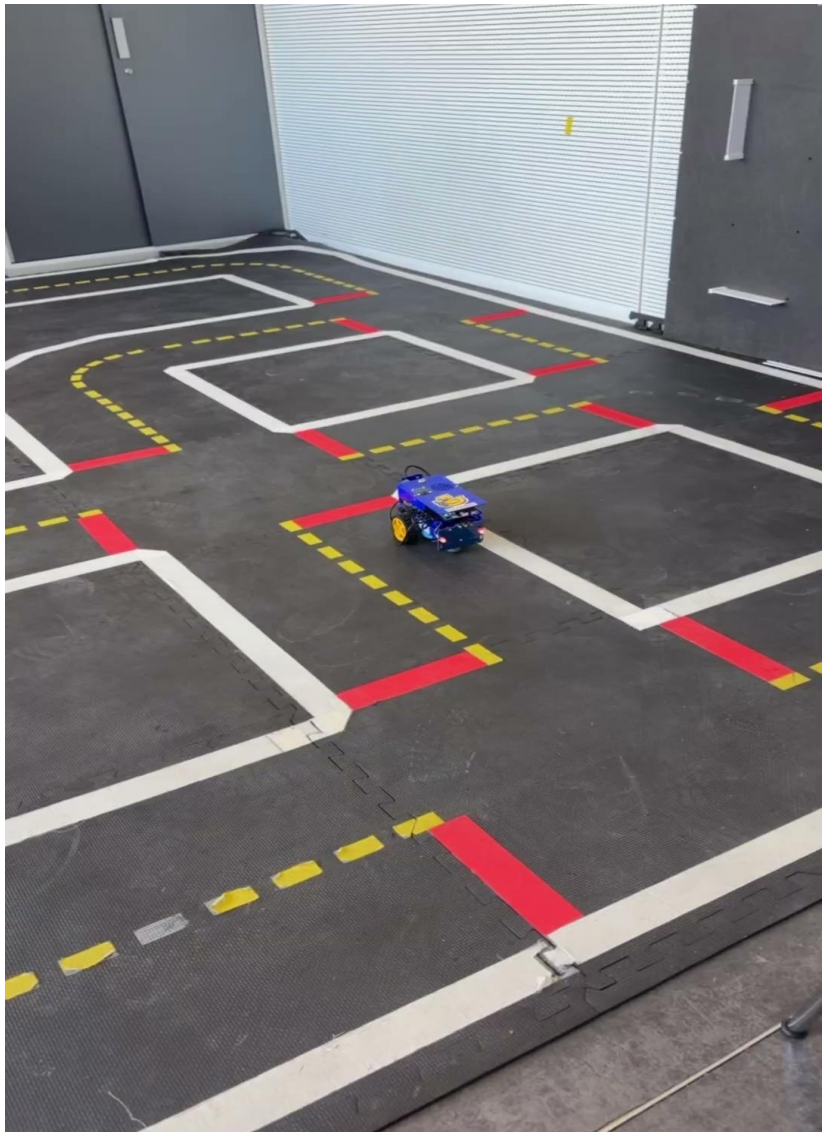
Goal:

Enable the Duckiebot to stay within the lane using visual input

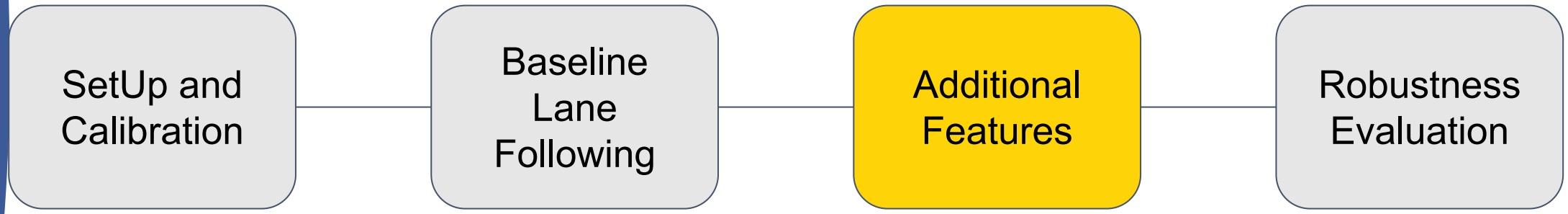
How it works:

- Image Capture: Camera captures road image
- Line Detection: Detect white and yellow lane markings
- Robot estimates its position relative to the lane
- Controller adjusts steering to stay centered
- Process runs continuously in real time

Lane Following

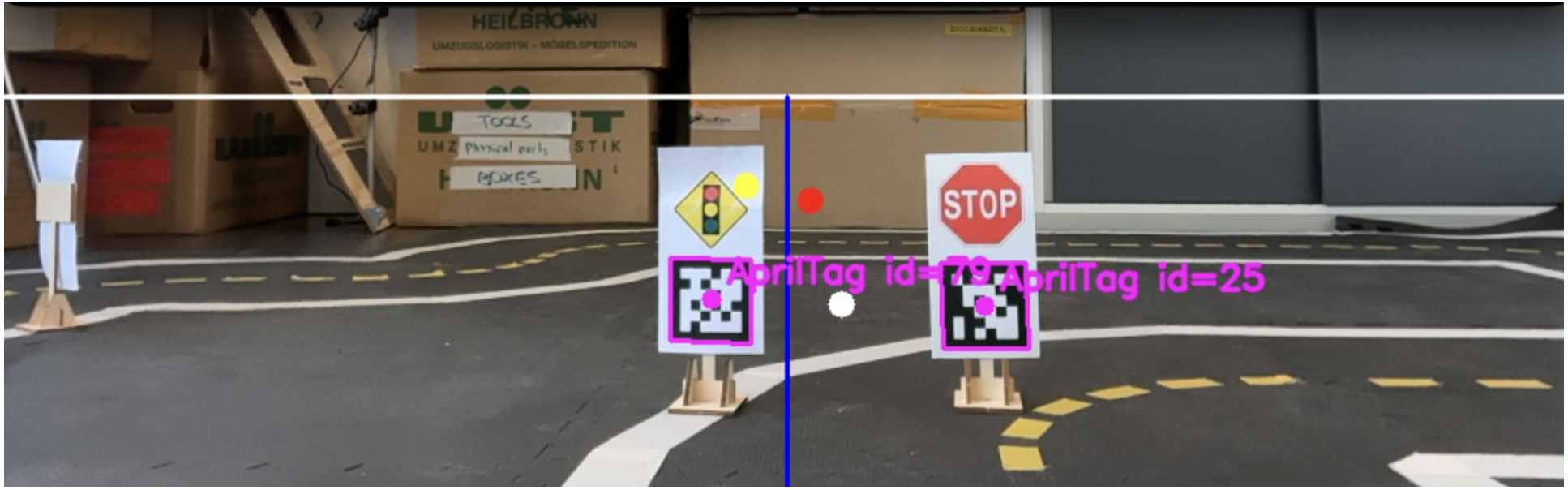


Milestones


SetUp and Calibration
Baseline Lane Following
Additional Features
Robustness Evaluation

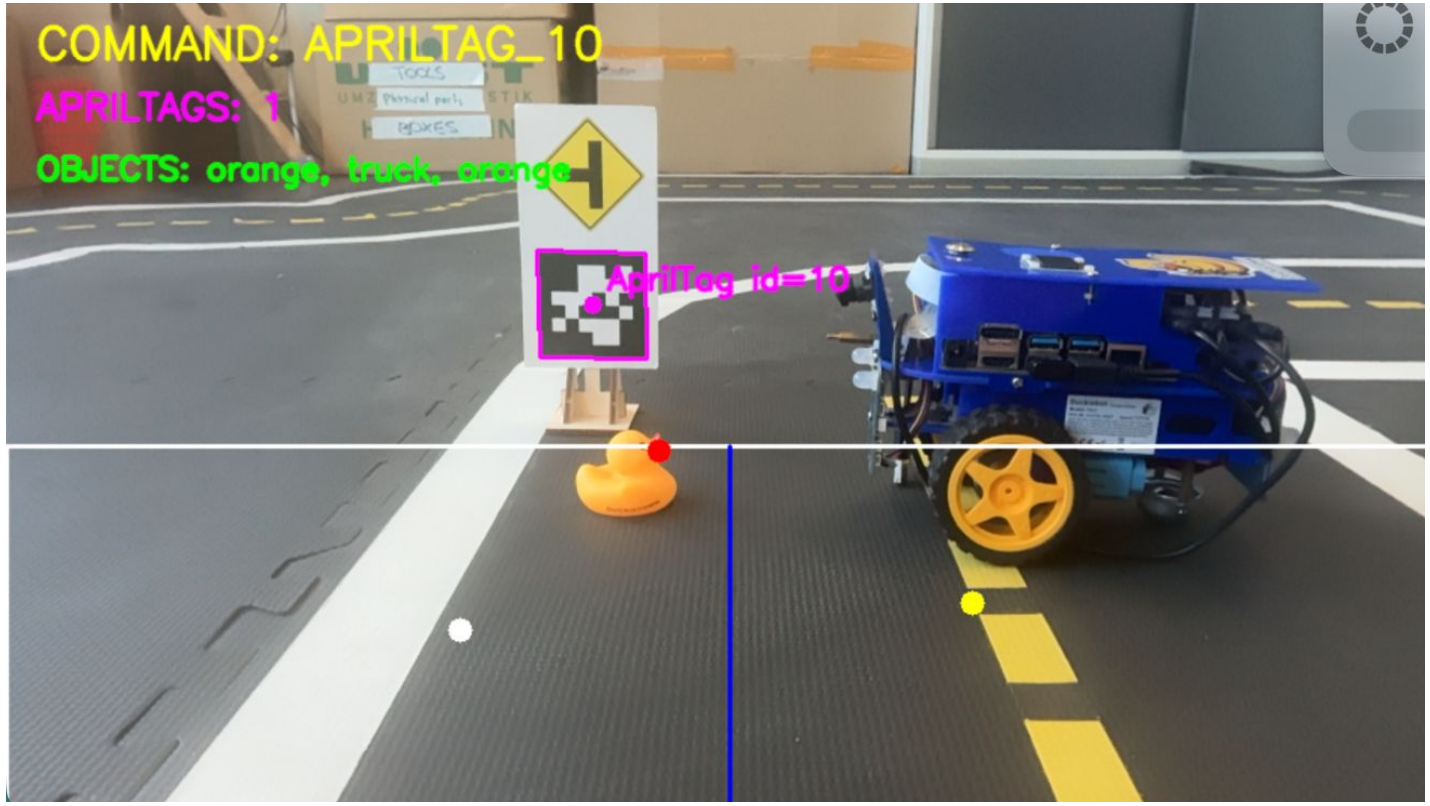
AprilTags

- Detects visual markers (AprilTags) in the environment
- Provides identification and position information
- Helps the robot recognize predefined signs

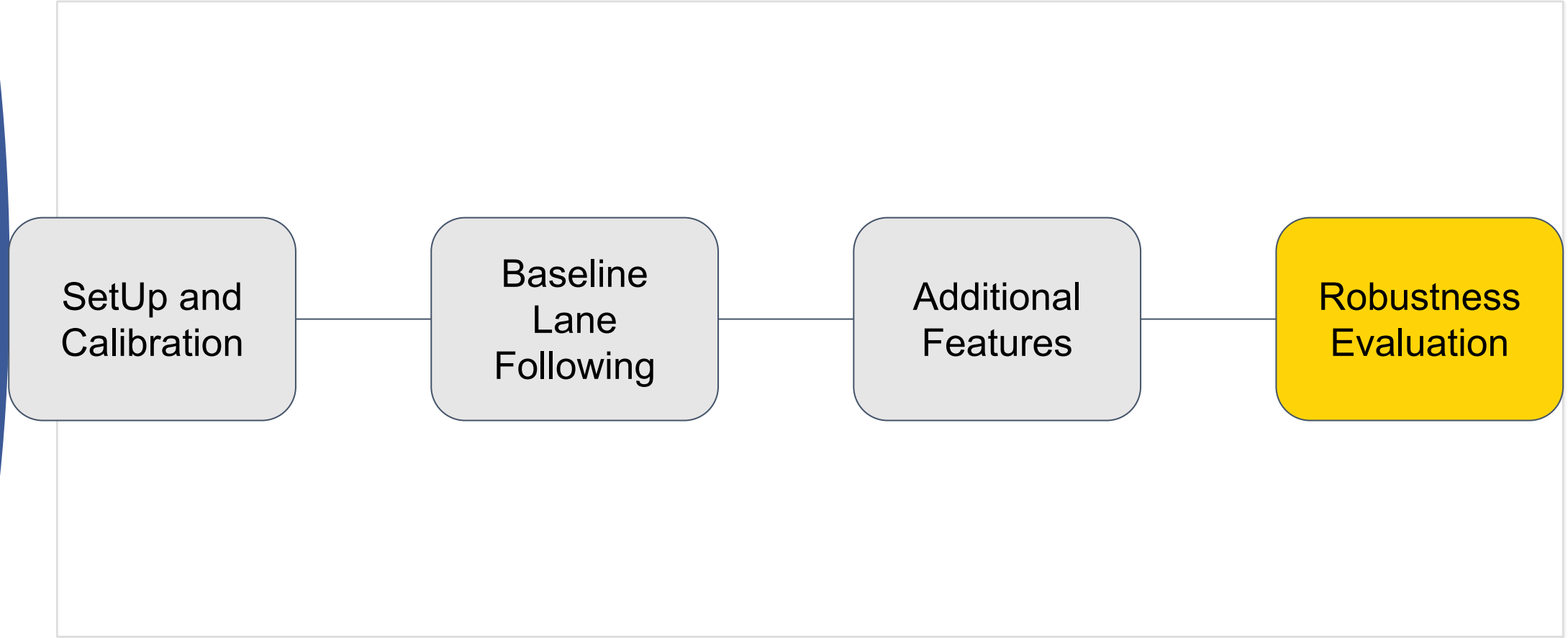


Object Detection

- Detects objects present on the road
- Uses visual features from camera input
- Enables reaction to obstacles



Milestones


SetUp and Calibration
Baseline Lane Following
Additional Features
Robustness Evaluation

Disturbances Tested- Lighting Change

Expected Influence:

- Reduced contrast between lane markings and road

Affected Modules:

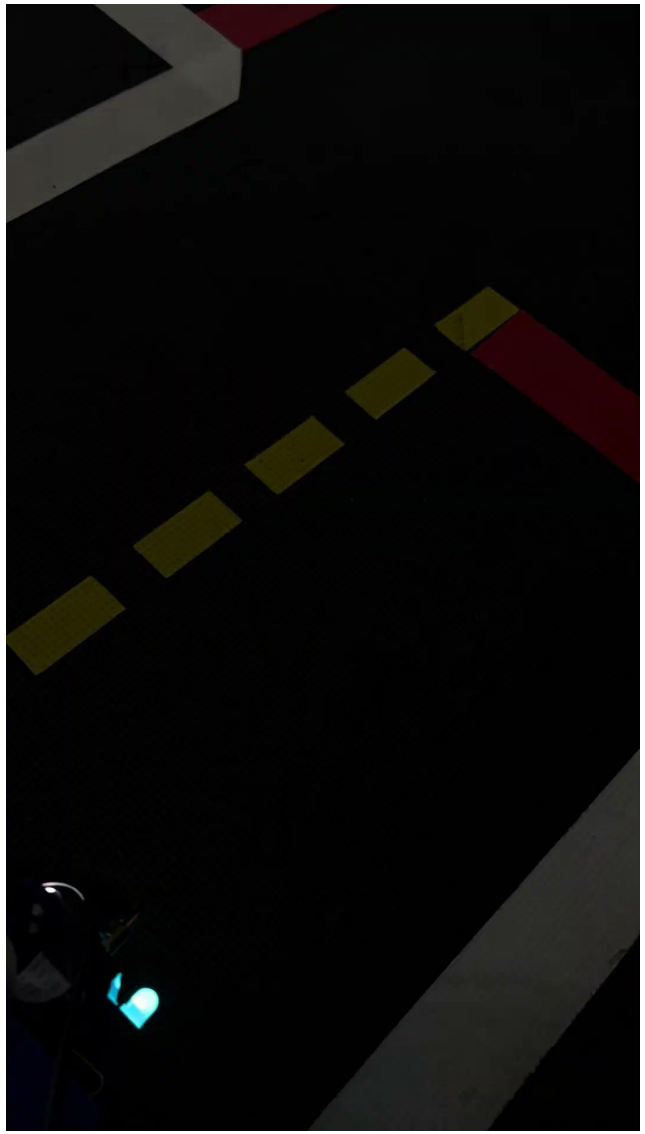
- Image Processing
- Line Detector

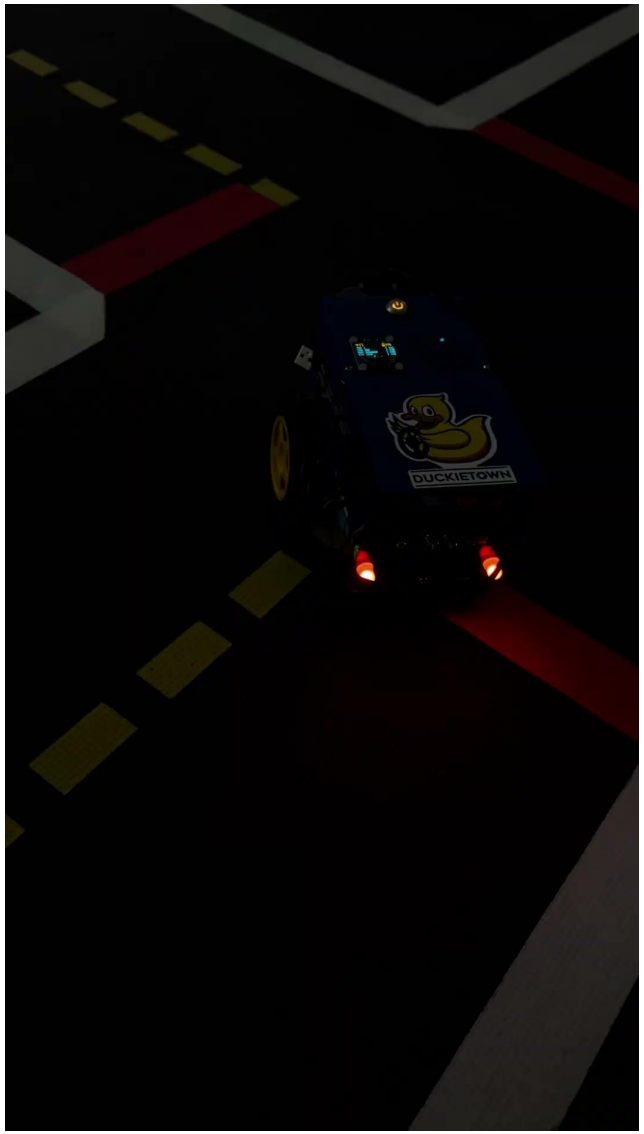
Perception Effect:

- Reduced or distorted segment detection
- Less reliable lane pose estimation

Behavior Effect:

- Increased steering corrections
- Instability in lane following





Disturbances Tested- Shadow

Expected Influence:

- Shadows misclassified as objects

Affected Modules:

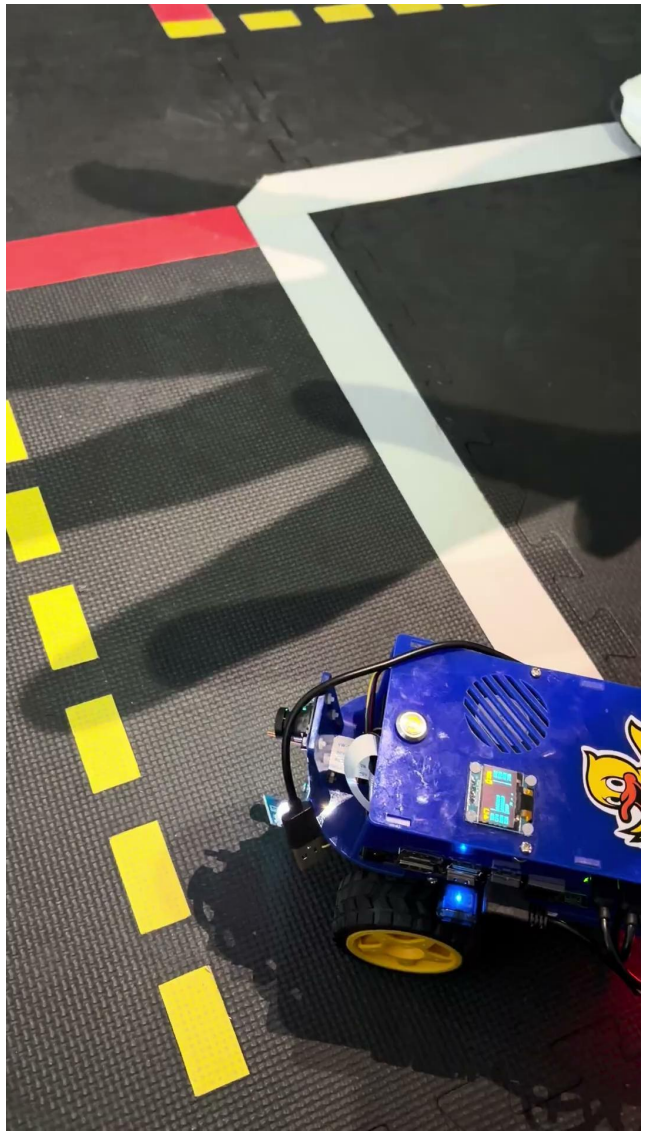
- Object Detection
- Visual Perception
- Control Logic

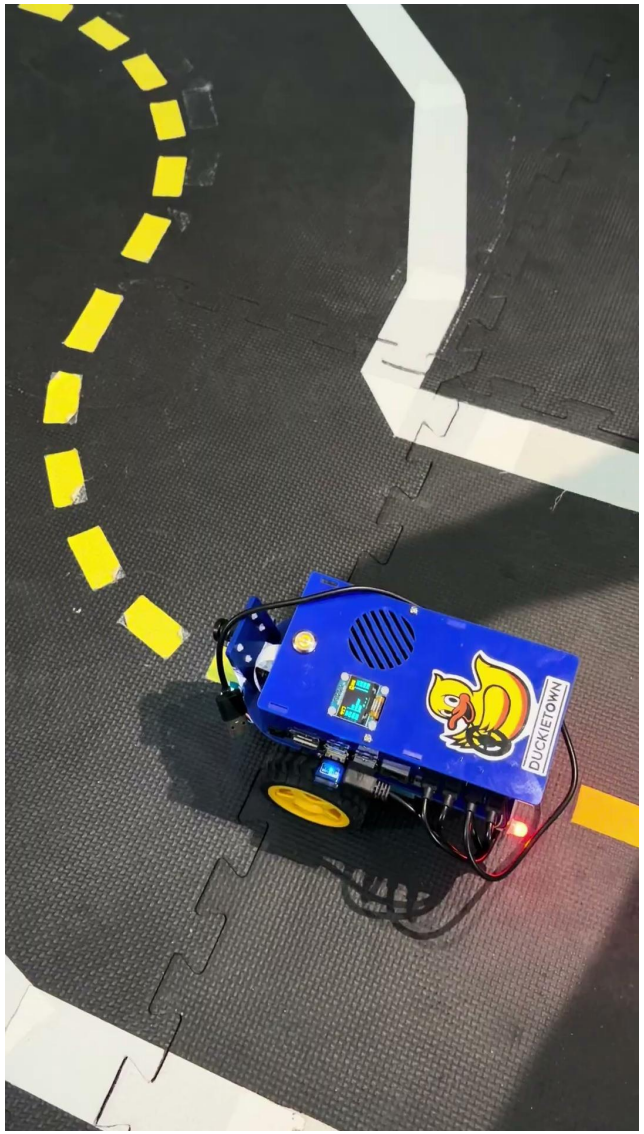
Perception Effect:

- False object detections
- Confusion in scene interpretation

Behavior Effect:

- Unnecessary stopping
- Incorrect reactions to non-existent obstacles





Disturbances Tested- Calibration Variation

Expected Influence:

- Geometric projection errors

Affected Modules:

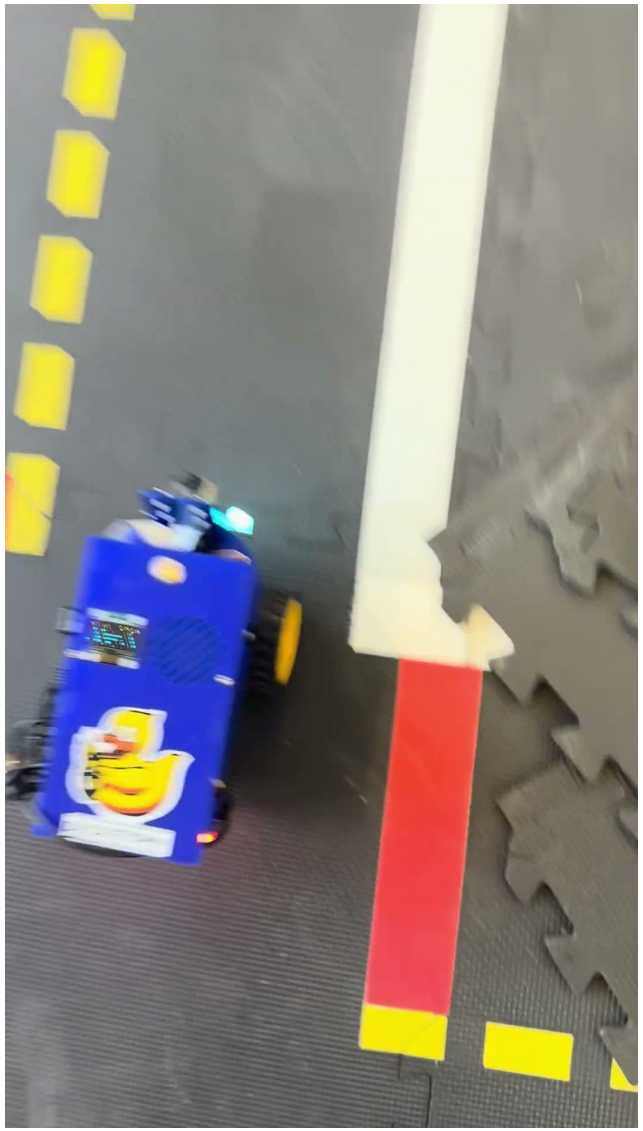
- Ground Projection
- Lane Filter

Perception Effect:

- Biased projected lane geometry
- Lane does not align with real track

Behavior Effect:

- Offset from lane center
- Incorrect steering tendency
- Does not follow Stop line detection



Observation and Key Insights

- System performed well under baseline conditions
- Lighting and shadows reduced detection reliability
- Shadows sometimes triggered false object detections
- Calibration errors caused consistent offset in lane following

- Perception robustness is critical for stable autonomous driving
- Errors in perception propagate through the entire pipeline
- Visual disturbances cause noisy outputs
- Calibration errors introduce systematic bias
- Reliable perception is essential for all extended functions

Challenges (1)

Image Architecture Compatibility

Problem:

The Duckietown Docker containers were built for **amd64 architecture**, but our **Ubuntu Virtual Machine was running on aarch64 (ARM64)** architecture.

Solution:

We retagged the ARM64 image so Duckietown would use the correct architecture image.

Commands Used :

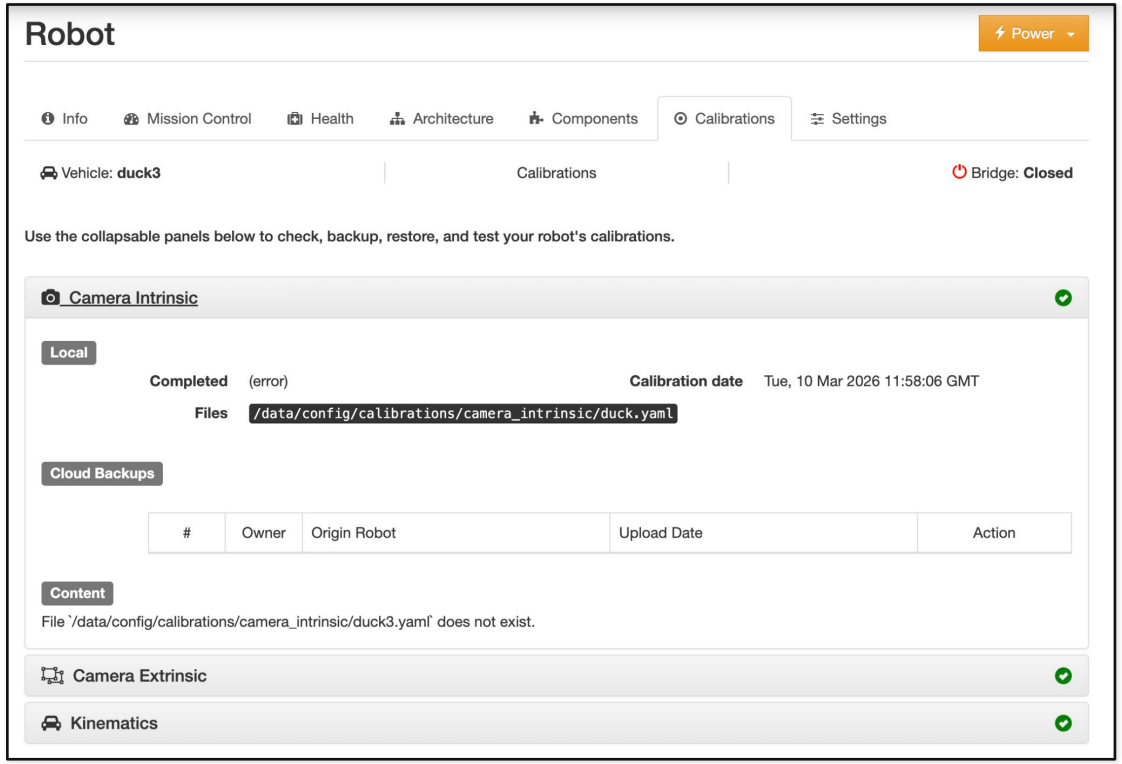
```
tannu@tannu: ~
tannu@tannu:~$ docker rm -f joystick_gui_duck 2>/dev/null
docker rmi duckietown/dt-gui-tools:daffy-amd64
docker tag duckietown/dt-gui-tools:daffy-arm64v8 duckietown/dt-gui-tools:daffy-amd64
xhost +local:docker
dts duckiebot keyboard_control duck
```

Challenges (2)

Robot Hostname Changed

Problem

- Robot hostname changed automatically: **duck** → **duck-2** → **duck3**
- Calibration files are saved under **duck**, so **duck3 cannot find calibration files and the robot cannot run**



Challenges (3)

3. Display Screen Stopped Working

Problem: Could not see status or IP , Debugging became difficult

Solution: Used SSH and dashboard instead of screen

4. Battery Issue

Problem: Robot shutting down randomly

Solution: Replaced battery

5. Camera Failure

Problem: Camera stopped giving image

Lane following and calibration failed

Solution: Replaced camera

Future Work

- Improve robustness under varying lighting conditions
- Reduce false detections caused by shadows
- Enhance object detection reliability
- Develop adaptive parameter tuning
- Combine multiple perception cues for better accuracy
- Implement more advanced decision-making strategies

Goal:

Make the system more reliable in real-world environments

Thank you!